

Web Hacking - Fresh Juice

A cybersteps project

Date: 24. November 2025

Participants: Ali, Max, Gregor



Executive summary:

The goal is to find as many vulnerabilities in the famous OWASP Juice Shop. An application, which is vulnerable by design and has 172 challenges to solve.

Preparation:

To run the application we use docker, which is a container application.

1. Install docker
2. Verify installation:

```
docker run hello-world
```

3. Download the Juice-shop-application and run it:

```
docker pull bkimminich/juice-shop  
docker run -d --name juice-shop -p 3000:3000 bkimminich/juice-shop
```

Findings:

We were able to solve 42/172 challenges (including coding Challenges).

This is a list of all found vulnerabilities:

#	Vulnerability	Category	Difficulty
1	Score Board	Miscellaneous	1 Star
2	Privacy Policy	Miscellaneous	1 Star
3	Bully Chatbot	Miscellaneous	1 Star
4	Error Handling	Security Misconfiguration	1 Star
5	Missing Encoding	Improper Input Validation	1 Star
6	Repetitive Registration	Improper Input Validation	1 Star
7	Zero Stars	Improper Input Validation	1 Star
8	Exposed credentials	Sensitive Data Exposure	2 Stars
9	Login Admin	Injection	2 Stars
10	Admin Section	Broken Access Control	2 Stars
11	View Basket	Broken Access Control	2 Stars
12	Empty User Registration	Improper Input Validation	2 Stars
13	Five-Star Feedback	Broken Access Control	2 Stars
14	Login MC SafeSearch	Sensitive Data Exposure	2 Stars
15	Meta Geo Stalking	Sensitive Data Exposure	2 Stars
16	Forged Feedback	Broken Access Control	3 Stars
17	CAPTCHA Bypass	Broken Anti Automation	3 Stars
18	Access Log	Observability Failures	4 Stars
19	Christmas Special	Injection	4 Stars
20	Easter Egg	Broken Access Control	4 Stars
21	Forgotten Sales Backup	Sensitive Data Exposure	4 Stars
22	GDPR Data Theft	Sensitive Data Exposure	4 Stars
23	Leaked Unsafe Product	Sensitive Data Exposure	4 Stars
24	Misplaced Signature File	Observability Failures	4 Stars
25	Nested Easter Egg	Cryptographic Issues	4 Stars
26	Poison Null Byte	Improper Input Validation	4 Stars
27	Steganography	Security through Obscurity	5 Stars
28	Email Leak	Sensitive Data Exposure	5 Stars

#	Vulnerability	Category	Difficulty
29	Reset Bjoern's Password	Broken Authentication	5 Stars

Detailed explanation:

1. Score board

Going through the source code, we could find a path (`/#/score-board`) in the `main.js`-file.

4. Error Handling

Category: Security Misconfiguration

Technique: T1190 – Exploit Public-Facing Application

Mitigation:

- **M1051 – Update/Patch Applications** – disable debug errors in production
- **M1050 – Secure Coding** – proper error handling

Prevention:

- Custom error pages
- Do not reveal stack traces

5. Missing Encoding

Category: Input Validation

Technique: T1059 – Command/Script Injection

Mitigations:

- **M1050 – Secure Input Validation and Output Encoding**

Prevention:

- Encode output before rendering in HTML

6. Repetitive Registration

On registration, when you type in matching passwords, but afterwards change the first one, you are able to registrate with not matching passwords. Also both passwords (password and repeated password) gets send to the server, which is not necessary.

Mitigation:

- **M1032 – MFA and account protections**
- **M1050 – Validate Inputs**

Prevention:

- Check for duplicate accounts
- Rate limiting

7. Zero Stars

When you intercept the filled out review form with burp, you can just change the given stars to 0.

Mitigation:

- **M1050 – Input validation server-side**
Prevention:
 - Enforce business logic rules server-side

8. Exposed credentials

Also in the main.js file there was the credentials 'testing@juice-sh.op / test123' hardcoded, which we could log in with.

Mitigation:

- **M1027 – Encrypt sensitive data**
- **M1016 – Vulnerability Scanning**
Prevention:
 - Never store credentials in plaintext or comments

9. Login Admin

Logging in as admin could be done in different ways. One way is to bruteforce the password, with the help of burpsuite, because it was a very weak username/password combination: "admin@juice/sh.op / admin123"

The second way was to login with SQL injection:

Putting ' -- after the username and inserting any password would let you log in.

Mitigation:

- **M1050 – Use prepared statements/ORM**
Prevention:
 - Parameterized queries
 - WAF rules

10. Admin Section

In the main.js, we could find a path to /administration. You need to be logged in as a admin to view this page. But since we already were able to log in as administrator, we could access the admin section.

Mitigation:

- **M1030 – RBAC and authorization checks**

Prevention:

- Enforce server-side authorization

11. View Basket

Intercept a basket-request with burp-suite, while being logged in as any user and change the user-ID.

Mitigation:

- **M1026 – Least privilege**

Prevention:

- Do not trust client-side checks

12. Empty User Registration

Intercepting the filled out registration-form with burpsuite, let you delete username and password and registrate a user with empty username and password.

Mitigation:

- **M1050 – Input validation & sanitization**

Prevention:

- Reject missing/malformed fields

13. Five-Star Feedback

Logging in as administrator (see Nr. 9) and delete all 5-Star-reviews.

Mitigation:

- **M1030 – Server-side authorization checks**

Prevention:

- Enforce user identity checks server-side

14. Login MC SafeSearch

Category: Sensitive Data Exposure

Technique: T1552

Mitigations:

- **M1027 – Encrypt sensitive info at rest and in transit**

Prevention:

- Secure session storage
- Use TLS everywhere

15. Meta Geo Stalking

First we looked in the login section what John's security question is.

The question was "What's your favorite place to go hiking?"

The task also gave the hint to find a clue on the photo-wall which is at `/#/photo-wall`.

There is a photo with the subtitles 'I love to go hiking here... @j0hnNy'.

Downloading this photo and feeding it into a online-metadata-extractor revealed geo-data, which pointed to a area in Kentucky/USA.

We created a list of area-, hiking-path-names of that particular area and ran a Burpsuite-Intruder attack which revealed that the answer to his security question was: Daniel Boone National Forest.

Mitigation:

- **M1027 – Mask/redact sensitive metadata**

Prevention:

- Sanitize uploaded files

16. Forged Feedback

To give a review as another user, all you had to do, was again to intercept a filled out review form and change the UserID.

Mitigation:

- **M1030 – Authorization enforcement**

Prevention:

- Verify user identity server-side

17. CAPTCHA Bypass

The solution was again to intercept a filled-out review form with a solved CAPTCHA. We then forwarded it to the Repeater and spammed these requests.

Mitigation:

- **M1017 – User training**
- **M1050 – Harden CAPTCHA system**

Prevention:

- Invisible reCAPTCHA
- Rate limiting

18. Access Log

Category: Observability Failure

Technique: T1005 – Data Collection

Mitigations:

- **M1051 – Patch logging configuration**

Prevention:

- Do not log sensitive information

19. Christmas Special

Category: Injection

Technique: T1190

Mitigations:

- **M1050**, input sanitization

Prevention:

- Validate input server-side

20. Easter Egg

As mentioned above, we found the path the ftp (/ftp). There you could find a 'eastere.gg'-file.

The server only allowed to download .pdf and .md files.

To access the eastere.gg-file you had to inject a nullbyte and url-encode it and add a allowed ending:

'/ftp/eastere.gg%2500.md' lets you download the file.

Mitigation:

- **M1030 – RBAC**

Prevention:

- Hide admin-only endpoints

21. Forgotten Sales Backup

Category: Data Exposure

Technique: T1530 – Data Leak

Mitigations:

- **M1029 – Remote data deletion**

Prevention:

- Do not store backups in web root

22. GDPR Data Theft

Category: Sensitive Data Exposure

Technique: T1537 – Exfiltration

Mitigations:

- **M1027 – Encrypt data at rest**
- **M1030 – Access segmentation**

23. Leaked Unsafe Product

Category: Sensitive Data Exposure

Technique: T1530

Mitigations:

- **M1027 – Secure storage**
- Prevention:**
- Implement data classification and ACL

24. Misplaced Signature File

Like # 20. Easteregg you can download any files from the ftp-server using the urlencoded nullbyte + .md or .pdf. One of it was a Misplaced Signature File.

25. Nested Easter Egg

Inside the eastere.gg-file, we found in challenge 20, was a string which was easily identifiable as a base64 string.

Putting that in Cyberchef and decode it revealed this string:

"/gur/qrif/ner/fb/shaal/gurl/uvq/na/rnfgre/rtt/jvguva/gur/rnfgre/rtt"

It looked like a url, but visiting did not work.

We suspected it might be a Caesar-cipher and tried ROT13 which resulted in:

"/the/devs/are/so/funny/they/hid/an/easter/egg/within/the/easter/egg"

Adding this to the url solved the challenge.

26. Poison Null Byte

Like described in # 20 and # 24. To access files on the ftp-server, with not allowed file-endings (everything but .md and .pdf) you need to add a Url-encoded nullbyte + a allowed file ending.

nullbyte = %00 > '%' in urlencode is %25 +00 = %2500

for example: coupons_2013.md.bak > coupons_2013.md.bak%2500.md

- **M1050 – Input sanitization**

Prevention:

- Validate uploaded file names

27. Steganography

Category: Security through Obscurity

Technique: T1095 – Hidden data tunneling

Mitigations:

- **M1016 – Scanning and review**

Prevention:

- Don't rely on obscurity for protection

28. Email Leak

Category: Sensitive Data Exposure

Technique: T1530

Mitigations:

- **M1027 – Mask sensitive user data**

29. Reset Bjoern's Password

Category: Broken Authentication

Technique: T1078 – Valid Accounts

Mitigations:

- **M1032 – MFA and secure reset flow**

Prevention:

- Use tokenized password reset
- ID verification